

Minuta — Tema 01: Conceptos Introdutorios + Intro a TypeScript

Estado: GENERADA **Agente:** Dr. Roberto (class-writer) **Fecha:** 2026-03-09 **Duración total:** 120 minutos **Perfil docente:** profesor-teorico **Semana / Clase:** Semana 1, Clase 1 de 2 **Input:** temas/01-conceptos-introductorios/disenio.md (aprobado) **Output previsto:** minuta.md + filminas.md

Antes de empezar — Preparación (□10 min antes de clase)

- Abrir Deno Playground: <https://playground.deno.land>
- Tener preparada la demo de IA (Copilot o Claude en el navegador)
- Verificar que el proyector muestra las filminas correctamente
- Comprobar que el snippet de C y el de TypeScript estén en el buffer del editor

Material de referencias rápidas en pizarrón (optativo): escribir los 4 paradigmas antes de que entren los alumnos — genera curiosidad.

APERTURA — Bienvenida a la materia (5 min, fuera del tiempo de bloques)

□ **Filmina en pantalla durante la apertura:** F-01 (portada) *Tono: cálido, situador. No empezar con teoría todavía.*

- Presentarse brevemente: nombre, cómo prefieren que le digan, años en la cátedra.
 - Decir explícitamente: *“Esta es una materia que parece rara al principio — ¿para qué estudiar lenguajes si ya saben programar? La clase de hoy responde esa pregunta.”*
 - Aclarar la dinámica de las clases: teoría con ejemplos, demos en vivo, preguntas bienvenidas.
 - Mencionar que el lenguaje principal este año es **TypeScript** (y por qué cambió).
-

BLOQUE 1 — ¿Por qué estudiar lenguajes de programación? (20 min)

□ **Filminas de este bloque:** F-02 · F-03 · F-04 · F-05 · F-06 · F-07 · F-08 · F-09 · F-10 · F-11 · F-12 · F-13

Objetivo del bloque: Justificar el valor de la materia desde un ángulo práctico y provocador.

Entrada al bloque (□ F-02) (2 min)

Plantear la pregunta disparadora directamente:

“¿Cuántos lenguajes de programación existen hoy?”

Dejar que respondan. La respuesta real: más de 700 con uso documentado, miles de dialects. Preguntar: > *“¿Tuvo sentido crear tantos? ¿O es un caos?”*

Transición: *“La respuesta corta es: cada lenguaje resuelve un problema que los otros no resolvían bien. Hoy vamos a ver cuáles son esos problemas y cómo los lenguajes los abordan.”*

Definición formal — qué es un lenguaje de programación (□ F-03) (2 min)

Antes de argumentar por qué estudiar lenguajes, precisar de qué se habla:

- **Cómputo:** una serie de operaciones estructuradas aplicadas a datos de entrada para obtener nuevos datos.
- **Programa:** una colección definida y ordenada de cálculos diseñada para realizar una tarea específica.
- **Lenguaje de programación:** un conjunto de reglas sintácticas y semánticas usadas para definir programas — sistema de notación cuyas instrucciones son ejecutadas por máquinas.

“Aaby, citado en Loudon & Lambert Cap. 1: ‘Una descripción completa incluye el modelo computacional, la sintaxis, la semántica y las consideraciones pragmáticas.’ — Esos tres ejes son el mapa de toda la materia.”

Transición rápida: *“Con eso claro, ¿por qué importa elegir bien el lenguaje?”*

El costo de elegir mal (□ F-04) (5 min)

Presentar el argumento económico con un caso real o hipotético:

- **Caso típico:** Una startup elige Node.js para un sistema de cálculo numérico intensivo. Dos años después migran a Python+NumPy con pérdida significativa de código. ¿Por qué pasó? Porque quien eligió el lenguaje no conocía los paradigmas.
- Más directo aún: la elección del lenguaje determina el modelo mental con el que se piensa el problema.

“No es solo sobre sintaxis. Es sobre cómo pensás el problema. Eso es un paradigma.”

Perspectiva histórica express (□ F-05) (7 min)

Guiar rápidamente la línea de tiempo — usar la filmina de timeline:

Hito	Lenguaje	¿Qué aportó?
1957	FORTRAN	Primer lenguaje de alto nivel — reemplazó ensamblador
1960	LISP	Funciones como datos, recursión — programación funcional
1972	C	Imperativo estructurado, portabilidad
1980	OO puro	Orientación a objetos pura
1995	Java	OO + máquina virtual □ “write once, run anywhere”
2008	Python 3	Multiparadigma, ecosistema IA
2012	TypeScript	Tipos estáticos sobre JS □ escalabilidad

índice de tiobe

“Cada punto en esta línea resuelve un problema del anterior. No es evolución lineal — es diversificación.”

Criterios de evaluación de lenguajes (□ F-06 · F-07 · F-08 · F-09 · F-10 · F-11 · F-12) (6 min)

“Si yo les pido que evalúen un lenguaje, ¿qué mirarían? Sebesta sistematizó esto.”

Presentar los 6 criterios (filmina):

1. **Legibilidad** — ¿Se puede leer código de otro programador? Python vs. Perl.
2. **Escribibilidad** — ¿El lenguaje facilita expresar la intención? SQL para consultas vs. ensamblador.

3. **Confiabilidad** — ¿Los tipos y verificaciones formales reducen bugs? TypeScript vs. JavaScript puro.
4. **Costo** — Desarrollo, mantenimiento, capacitación del equipo.
5. **Portabilidad** — ¿Funciona en distintas plataformas sin reescribir?
6. **Eficiencia** — Velocidad de ejecución y consumo de recursos.

Ejemplo de tensión: > “Python tiene alta legibilidad y escribibilidad, pero baja eficiencia. C tiene alta eficiencia pero menor legibilidad. No hay lenguaje perfecto — hay trade-offs.”

Punto de tensión para discusión rápida (□ F-13) (2 min): > “¿Importa el lenguaje si la IA puede escribir en cualquiera? ¿O precisamente por eso importa más saber evaluarlos?” No resolver ahora — dejar abierto. **Retomar explícitamente al inicio del Bloque 5:** “¿Se acuerdan de la pregunta que dejamos abierta? Ahora la respondemos.”

BLOQUE 2 — Los paradigmas: mapa general (25 min)

□ **Filminas de este bloque:** F-14 · F-15 · F-16 · F-17 · F-18 · F-19 · F-20

Objetivo del bloque: Dar el vuelo de 10.000 pies sobre el mapa de paradigmas con base teórica sólida.

¿Qué es un paradigma? (□ F-14) (4 min)

“Un paradigma de programación es una forma de pensar el cómputo. Es el modelo mental que estructura cómo describís la solución.”

No es solo sintaxis — dos lenguajes con sintaxis muy distintas pueden compartir paradigma (Java y C# son ambos OO). Un lenguaje puede implementar varios paradigmas (TypeScript).

Los factores históricos que formaron los paradigmas (□ F-15 · F-16 · F-17) (6 min)

“Los paradigmas no surgieron de la nada — fueron respuestas a restricciones de hardware y metodología.”

La arquitectura de Von Neumann □ paradigma imperativo:

- La CPU lee instrucciones de la memoria y las ejecuta secuencialmente.
- Las “variables” en un programa imperativo mapean directamente a celdas de memoria.
- Esto no es una decisión de diseño — es una limitación heredada del hardware de los años 40.
- La pregunta que surge: ¿hay otra forma de describir cómputo que no dependa de este modelo?

La evolución metodológica (cronología comprimida):

- Hasta los 70: programación “artesanal” — sin metodología sistemática
- 70s: análisis y diseño estructurado — modularización, eliminación del GOTO
- Abstracción de datos (Simula, Ada) — el dato y sus operaciones juntos
- Programación funcional (LISP) — sin estado, funciones como objetos
- OO — objetos que intercambian mensajes
- Multiparadigma (Python, TypeScript, Scala) — lo mejor de todos

El cuello de botella de Von Neumann (□ F-18) (5 min)

“Louden y Lambert plantean algo interesante en el Cap. 1: la ejecución secuencial instrucción a instrucción no es una verdad universal — es una restricción del hardware.”

- Límite: la velocidad del bus CPU □ memoria es el cuello de botella

- Esto limita el paralelismo y el cómputo no determinista
- **Pregunta disparadora:** “¿Se puede describir cómputo sin depender de Von Neumann?”> “No es solo sobre sintaxis. Es sobre cómo pensás el problema. Eso es un paradigma.”
- Respuesta: sí — y de ahí nacen el funcional (lambda calculus) y el lógico (lógica simbólica)

Los 4 paradigmas fundamentales (□ F-19) (8 min)

Presentar la tabla comparativa (filmina dedicada):

Paradigma	Base formal	Unidad	Estado	Ejemplos
Imperativo	Máquina de Von Neumann	Instrucción	Mutable	C, Go, Pascal
OO	Imperativo + encapsulamiento	Objeto / mensaje	Mutable (encapsulado)	Java, C#, Dart
Funcional	Cálculo lambda (Church, 1936)	Función	Inmutable	Haskell, Clojure, LISP
Lógico	Lógica simbólica (resolución)	Relación / hecho	Sin concepto de estado	Prolog

“Noten que el OO es una extensión del imperativo — no nació de cero. El funcional y el lógico tienen raíces matemáticas pre-computadoras.”

Multiparadigma: TypeScript, Python, Scala, F# — no eligen uno, los combinan. Esto tiene ventajas (flexibilidad) y desventajas (inconsistencia de estilo en equipos).

Lenguajes puros vs. multiparadigma + dominios de aplicación (□ F-20) (2 min)

Rápido, para conectar con el tema que viene:

- Haskell: funcional puro □ finanzas de alta confiabilidad, compiladores
- Prolog: lógico puro □ IA simbólica, sistemas expertos
- C: imperativo puro □ sistemas embebidos, kernels, drivers
- TypeScript: multiparadigma □ web full-stack, APIs, tooling moderno

BLOQUE 3 — Paradigma imperativo y máquina abstracta (20 min)

□ Filminas de este bloque: F-21 · F-22 · F-23 · F-24

Objetivo del bloque: Conectar la teoría de máquinas abstractas con código real. Dar rigor sin perder claridad.

La escalera de abstracciones (□ F-21) (5 min)

“Gabbrielli plantea esto con precisión: todo lenguaje define una máquina abstracta. No hay lenguaje sin máquina.”

Usar la metáfora de la escalera (filmina):

LC-3 (ensamblador)	– Nivel 0: legibilidad nula, completo control
□ abstracción	
C, Pascal, Go	– Nivel 1: estructura, tipos, funciones
□ abstracción	

Java, Python, TypeScript – Nivel 2: GC, tipado, ecosistema

□ abstracción

React, Django, Rails – Nivel 3: frameworks – abstracción de la plataforma

“Al subir, gano expresividad y legibilidad. Al bajar, gano control y eficiencia. La pregunta clave: ¿qué abstraigo y qué pierdo?”

La conexión Von Neumann □ código imperativo (□ F-22) (5 min)

Conectar concretamente:

- Cada **variable** = celda de memoria con nombre
- La **asignación** `x = 5` = transferencia de valor CPU □ memoria
- Los **saltos de control** (if, while) = saltos condicionales/incondicionales del procesador
- El **estado** de un programa = conjunto de pares (nombre, valor) de todas las variables en un instante
- Un **cómputo** = una sucesión de estados □ el efecto de cada instrucción es modificar el estado

“El paradigma imperativo no es una convención de programadores — es una abstracción directa de cómo funciona el hardware.”

Ejemplo comparativo: el mismo algoritmo en 3 niveles (□ F-23) (8 min)

Enunciado: Calcular la suma de los valores absolutos de un array de 10 enteros.

Nivel 0 — Ensamblador LC-3 (mostrar brevemente en filmína, no detenerse):

```
; Suma valores absolutos – LC-3
; R0 = acumulador, R1 = puntero, R2 = contador
    AND R0, R0, #0      ; acc = 0
    LEA R1, DATA       ; puntero inicio del array
    AND R2, R2, #0
    ADD R2, R2, #10      ; contador = 10
LOOP  LDR R3, R1, #0     ; cargar elemento
      BRzp POS          ; si >= 0, saltar a POS
      NOT R3, R3         ; negarlo (complemento a 2)
      ADD R3, R3, #1
POS   ADD R0, R0, R3      ; acc += abs
      ADD R1, R1, #1     ; avanzar puntero
      ADD R2, R2, #-1    ; contador--
      BRp LOOP          ; si queda, repetir
```

“13 líneas. Un programador sin comentarios no puede leer esto en 30 segundos.”

Nivel 1 — C imperativo puro:

```
int suma_abs(int arr[], int n) {
    int acc = 0;
    for (int i = 0; i < n; i++)
        acc += (arr[i] < 0) ? -arr[i] : arr[i];
    return acc;
}
```

“¿Qué ganamos? Legibilidad. ¿Qué sigue implícito? La gestión de punteros, los efectos en memoria. Seguimos siendo imperativos — variable acc, mutación en el loop.”

Discusión (2 min): ¿qué haría TypeScript diferente? (anticipación del Bloque 4)

Máquina abstracta, interpretación y compilación (□ F-24) (2 min)

Sintetizar el concepto de Gabbrielli Cap. 1:

- Todo lenguaje define una **ML** (máquina abstracta del lenguaje L)
- **Interpretación pura**: el intérprete decodifica y ejecuta en runtime □ flexible, más lento
- **Compilación pura**: traduce el programa completo a lenguaje objeto antes de ejecutar □ rápido, menos flexible
- **En la práctica**: siempre hay un **lenguaje intermedio** (la máquina real no es pura)

“TypeScript es exactamente esto en el siguiente bloque — van a ver el pipeline completo.”

BLOQUE 4 — Intro a TypeScript como lenguaje multiparadigma (30 min)

□ **Filminas de este bloque**: F-25 · F-26 · F-27 · F-28 · F-29 · F-30 · F-31 · F-32 · F-33 · F-34 · F-35 · F-36 · F-37 · F-38 · F-39

Objetivo del bloque: TypeScript como ejemplo vivo de los conceptos teóricos. Primera escritura de código de la cursada.

Paradigma funcional en profundidad (□ F-25 · F-26 · F-27 · F-28) (4 min)

“En TypeScript podés escribir funcional, pero hay lenguajes donde el funcional es la única opción — y eso tiene ventajas concretas.”

(□ F-25) — **Cálculos sin efectos secundarios**: - El cómputo se define como evaluación de expresiones, no como secuencia de instrucciones. - Las funciones son **puras**: dado el mismo input, siempre producen el mismo output — sin efectos secundarios. - El estado no cambia: no hay variables que se reasignen. - Ventajas: razonamiento matemático directo, paralelismo trivial, tests más predecibles.

(□ F-26) — **Composición de funciones**: - Construir programas como cadenas de transformaciones: $f(g(h(x)))$. - En TypeScript: `arr.map(f).filter(g).reduce(h)` — cada paso transforma los datos sin mutar.

“La composición reemplaza el estado: en lugar de ‘modificar esta variable paso a paso’, ‘aplicar estas funciones en secuencia’.”

(□ F-27) — **Estructuras inmutables**: - “Cambiar sin mutar”: crear una copia con el cambio, no modificar el original. - `const nuevoArr = [...arr, nuevoElemento]` en lugar de `arr.push(nuevoElemento)`.

(□ F-28) — **TypeScript permite funcional, pero no lo garantiza**: - Lenguajes como Haskell o Clojure **imponen** la inmutabilidad por diseño. - TypeScript la permite (`const`, `readonly`) pero no la fuerza — depende de la disciplina del programador.

Paradigma lógico — introducción (□ F-29 · F-30 · F-31 · F-32) (4 min)

“El paradigma lógico es el más diferente de todos — no hay instrucciones, no hay funciones, solo relaciones y preguntas.”

(□ F-29) — **Programar declarando “qué”, no “cómo”**: - En lógica simbólica, un programa es un conjunto de **hechos** y **reglas** sobre el mundo. - El motor de inferencia determina cómo llegar a la respuesta. - Lenguaje representativo: **Prolog** — base de sistemas expertos, IA simbólica, procesamiento de lenguaje natural.

(□ F-30) — **Hechos y reglas en Prolog**:

```
% Hechos: relaciones conocidas
padre(juan, maria).
padre(juan, pedro).
madre(ana, maria).
```

```
% Regla: X es progenitor de Y si X es padre o madre de Y
progenitor(X, Y) :- padre(X, Y).
progenitor(X, Y) :- madre(X, Y).
```

“Este código no dice ‘cómo buscar’ — dice ‘qué es verdad’. El motor deduce el resto.”

(□ F-31) — Hacer preguntas al sistema:

```
?- progenitor(juan, maria).    % Consulta: ¿juan es progenitor de maria?
true.
```

```
?- progenitor(X, maria).      % Consulta: ¿quién es progenitor de maria?
X = juan ;
X = ana.
```

“Nota la inversión: en lugar de ‘ejecutar una función’, ‘hacer una pregunta’. El sistema busca todas las respuestas posibles.”

(□ F-32) — ¿Para qué sirve la programación lógica? Pasar rápido — pregunta socrática para reflexión: > “¿Se imaginan un dominio donde describirle relaciones a la máquina sea más natural que darle instrucciones paso a paso?” - Sistemas expertos médicos: “si el paciente tiene fiebre Y tos □ posible diagnóstico X” - NLP: razonamiento sobre gramáticas como reglas - Motores de reglas de negocio: compliance, seguros, legales

¿Por qué TypeScript en 2026? (□ F-33 · F-34) (4 min)

Argumentos rápidos:

- **Ecosistema:** npm + 2.3 millones de paquetes, frontend + backend (Node/Deno/Bun)
- **Tipos:** el sistema de tipos más expresivo de los lenguajes mainstream
- **IA:** los modelos de lenguaje generan TypeScript con alta fidelidad □ más fácil auditar
- **Multiparadigma:** podemos escribir el mismo problema en estilo imperativo, funcional u OO en el mismo archivo

Mencionar el cambio curricular: “Antes usábamos Kotlin. TypeScript cumple los mismos objetivos conceptuales y tiene mayor adopción en el mercado 2025-2026.”

TypeScript como ejemplo de máquina intermedia (□ F-35) (8 min)

“¿Recuerdan lo que dijo Gabbrielli sobre lenguajes intermedios? Acá está en vivo.”

Dibujar el pipeline en pizarrón o mostrar filmína:

```
archivo.ts
  □ [tsc – compilador TypeScript]
archivo.js □ LENGUAJE INTERMEDIO
  □ [V8 / Node.js / Deno – intérprete JIT]
Ejecución en CPU
```

Comparar con otros:

```
Java:   .java □ [javac] □ .class (bytecode) □ [JVM]           □ ejecución
Python: .py   □ [CPython (implícito)] □ bytecode (.pyc) □ [CPython VM] □ ejecución
```

“No es ‘interpretado’ ni ‘compilado’ en sentido puro. Usa **máquina intermedia** — exactamente como predice Gabbrielli Cap. 1.”

¿Por qué importa?

- El compilador tsc hace verificación de tipos **antes** de ejecutar → más confiabilidad
- El lenguaje intermedio (JS) es el que realmente ejecuta la CPU (via V8) → el runtime es diferente del lenguaje que leen

Demo: el mismo problema en TypeScript — imperativo y funcional (■ F-36 · F-37) (10 min)

■ Setup Deno Playground — paso a paso:

1. Abrir <https://playground.deno.land> en el navegador del proyector
2. El editor tiene código de ejemplo por defecto — borrarlo todo con Ctrl+A → Delete
3. Pegar (o tipear) el código de abajo — tipear es más didáctico si el tiempo lo permite
4. Ejecutar con el botón ■ **Run** en la barra superior, o Ctrl+Enter
5. El output aparece en el panel inferior derecho

Fallback si Deno Playground no carga: <https://www.typescriptlang.org/play> (TypeScript Playground oficial — mismo resultado para estos ejemplos)

Paso 1 — Versión imperativa (■ F-36) (para comparar directamente con C):

```
function sumaAbs(arr: number[]): number {  
  let acc = 0;  
  for (let i = 0; i < arr.length; i++) {  
    acc += arr[i] < 0 ? -arr[i] : arr[i];  
  }  
  return acc;  
}
```

```
console.log(sumaAbs([3, -1, 4, -1, 5]));
```

■ **Output esperado:** 14

“¿Qué cambió respecto de C? Tipos anotados (: number[], : number), pero el esqueleto es idéntico — variable acc mutable, loop con índice. Mismo paradigma imperativo.”

Paso 2 — Versión funcional (■ F-37) (sin mutación, sin loop explícito):

```
const sumaAbs = (arr: number[]): number =>  
  arr.map(x => Math.abs(x))  
    .reduce((acc, x) => acc + x, 0);
```

```
console.log(sumaAbs([3, -1, 4, -1, 5]));
```

■ **Output esperado:** 14

“¿Qué cambió? Sin let, sin mutación, sin loop. map y reduce son funciones de orden superior — reciben funciones como argumento. Esto es programación funcional.”

Conectar con Louden & Lambert: > “El mismo patrón existe en Scheme (dialecto de LISP): (reduce + (map abs lista)). La idea tiene 60 años. TypeScript la adopta como ciudadano de primera clase.”

Pregunta para los alumnos: “¿Cuál les resulta más legible? ¿Cuál les parece más fácil de escribir? ¿Coinciden?” — Registrar respuestas brevemente en pizarrón.

Sistema de tipos básico de TypeScript (□ F-38) (8 min)

“TypeScript agrega una capa que JavaScript no tiene: **tipos estáticos**. Esto cambia la confiabilidad.”

□ **Continuar en Deno Playground** — pegar este código en un nuevo archivo o agregar debajo del ejemplo anterior.

Tipado básico en vivo:

```
// Type annotations explícitas
let nombre: string = "Paradigmas";
let año: number = 2026;
let activo: boolean = true;

// Inferencia – TypeScript deduce el tipo
let materia = "Paradigmas"; // tipo: string (inferido)

// Funciones tipadas
function saludar(nombre: string): string {
  return `Hola, ${nombre}!`;
}

// Arrays y objetos
const notas: number[] = [8, 9, 7, 10];

interface Alumno {
  nombre: string;
  legajo: number;
  notas: number[];
}
```

Mostrar el error en vivo:

```
// Error: Argument of type 'string' is not assignable to parameter of type 'number'
sumaAbs(["hola", "mundo"]); // tsc detecta esto antes de ejecutar
```

□ **Output esperado en el editor:** subrayado rojo en ["hola", "mundo"] con el mensaje: Argument of type 'string[]' is not assignable to parameter of type 'number[]'

En Deno Playground el error aparece en el panel inferior al intentar ejecutar.

“El compilador actúa como primer revisor. Esto es confiabilidad — criterio #3 de Sebesta.”

Acelerador de paradigma (□ F-39):

- TypeScript no obliga a un paradigma — puede escribirse de forma imperativa, funcional u OO
- La elección es del programador □ más responsabilidad, más libertad
- A lo largo de la materia van a ver los tres estilos con este mismo lenguaje

BLOQUE 5 — IA Generativa y los paradigmas (15 min)

□ **Filminas de este bloque:** F-40 · F-41 · F-42 · F-43 · F-44 · F-45

Objetivo del bloque: Conectar los fundamentos de la materia con el contexto actual de la IA. Motivar desde lo relevante para su futuro profesional.

El cambio de rol del programador (□ F-40) (4 min)

Cierre de F-13: *“En el Bloque 1 dejamos una pregunta abierta: ¿importa el lenguaje si la IA puede escribir en cualquiera? Ahora la respondemos — y la respuesta es que importa más que nunca, pero por razones distintas a las de antes.”*

Presentar los datos de Schmidt & Runfola (2025) — filmina con el split de tiempo:

Antes (pre-IA):

- 70% codificación manual + depuración
- 30% comprensión del problema

Hoy (con IA generativa):

- 20% codificación manual
- **50% prompting, supervisión y orquestación de IA**
- 30% formulación del problema

“Natural language has become the new compiler.”

“Lo que cambió no es que la IA escriba código — es que ahora el trabajo intelectual es decirle qué problema tiene que resolver y verificar que lo resolvió bien. Para eso necesitás saber paradigmas.”

La jerarquía de proficiencia en IA (□ F-41) (3 min)

Filmina con los 3 niveles (Schmidt & Runfola, Fig. 12):

- **AI Literacy:** cualquiera puede leer e interpretar lo que genera la IA
- **AI Fluency:** diseñar prompts, criticar el comportamiento del modelo, construir soluciones — **requiere pensamiento computacional**
- **AI Mastery:** construir, optimizar y auditar los sistemas que todos usan — para investigadores e ingenieros de IA

“Ustedes apuntan a AI Fluency. No llegan sin conocer paradigmas y semántica — la IA genera código en múltiples estilos y si no sabés distinguir uno de otro, no podés verificar lo que te da.”

Demo en vivo — La IA elige paradigmas (□ F-42 · F-43 · F-44) (6 min)

□ **Setup:** Abrir **Copilot Chat** (VS Code, panel lateral) o **Claude** (<https://claude.ai>) en el navegador del proyector. No hace falta configuración previa — basta con una sesión abierta.

Prompt 1 (□ F-42) — sin restricción de paradigma:

Escribí en TypeScript una función que calcule los primeros N números de la serie de Fibonacci usando mutación de variables (`let`) y un bucle.

□ Output típico (la IA tiende al imperativo):

```
function fibonacci(n: number): number[] {  
  let result: number[] = [];  
  let a = 0, b = 1;  
  for (let i = 0; i < n; i++) {  
    result.push(a);  
    [a, b] = [b, a + b];  
  }  
  return result;  
}
```

□ Señalar en vivo: `let result, let a, let b` mutables + `loop for` = **paradigma imperativo**. La IA por defecto tiende al paradigma más difundido entre la mayoría de usuarios.

Prompt 2 (□ F-43) — restricción funcional explícita:

Implementá lo mismo (serie de Fibonacci) en estilo funcional puro, sin mutación de estado, sin variables intermedias.

□ **Output esperado (funcional correcto):**

```
const fib = (n: number): number =>
  n <= 1 ? n : fib(n - 1) + fib(n - 2);
const fibonacci = (n: number): number[] =>
  Array.from({ length: n }, (_, i) => fib(i));
```

□ Verificar: sin `let`, sin `loop`, recursión pura. □ Si la IA usa `let` igual □ señalarlo como ejemplo de que la IA no siempre respeta las restricciones del prompt — requiere conocimiento de dominio para detectarlo.

Prompt 3 (□ F-44) — verificación de comprensión de máquinas abstractas:

Escribí en TypeScript una función que calcule los primeros N números de la serie de Fibonacci usando ``let`` y un bucle.

Luego explicá qué máquina abstracta ejecuta ese código.

□ **Respuesta correcta esperada** (verificar estos puntos en vivo):

- `tsc` compila `.ts` □ `.js` (lenguaje intermedio)
- V8 / Node.js / Deno ejecuta el `.js` resultante vía JIT
- TypeScript no se ejecuta directamente — siempre hay máquina intermedia
- Comparación con JVM (Java) o CPython (Python)

Si la IA omite alguno de estos puntos □ señalarlo como gap concreto del “trust but verify” en acción.

El loop “trust but verify” (□ F-45) (2 min)

Esquema rápido (filmina):

1. Formular el problema con precisión
2. Hacer el prompt a la IA
3. Revisar con conocimiento de dominio □ “¿qué paradigma usó? ¿es correcto semánticamente?”
4. Testear con casos borde
5. Refinar el prompt o escribir manualmente si falla

“El ‘sweet spot’: demasiada dependencia en la IA atrofia habilidades cognitivas. Los fundamentos son el antídoto.”

CIERRE (10 min)

□ **Filminas de este bloque:** F-46 · F-47

Mapa conceptual de la materia (□ F-46) (4 min)

Mostrar cómo se conectan los 15 temas del plan (filmina de mapa):

- **Tema 1-2:** Fundamentos — lenguajes, paradigmas, sintaxis, semántica
- **Temas 3-6:** Paradigma funcional (TypeScript + Python)
- **Tema 7:** Paradigma lógico (Prolog)

- **Temas 8:** Paradigma OO (TypeScript)
- **Temas 9-14:** Conceptos transversales — tipos, variables, control, módulos, polimorfismo
- **Tema 15:** Concurrencia y paralelismo

“Todo lo que vimos hoy es el mapa. Las próximas 15 clases son el territorio.”

Adelanto Clase 2 — Sintaxis y Semántica (□ F-47) (4 min)

“La próxima clase respondemos una pregunta que parece simple pero no lo es: ¿qué es un programa correcto?”

Plantear el disparador:

```
if (x !== 0) y = 1 / x;
```

*“¿Qué pasa si no ponemos el else? La **semántica** del lenguaje lo define — no la sintaxis. La diferencia entre sintaxis y semántica es la diferencia entre ‘esto es un programa válido’ y ‘esto hace lo que queremos’. Louden y Lambert, Cap. 1 §1.4.”*

- **Para la próxima:** instalar TypeScript o acceder a Deno Playground — vamos a escribir código
- Referencia: Deno Playground · TypeScript Playground

Cierre de clase (2 min)

“Resumen en una frase: un paradigma es un modelo mental para describir cómputo; cada lenguaje implementa uno o varios; TypeScript los implementa todos; y entender esto es la diferencia entre pedirle bien a la IA y recibir basura.”

Preguntas de sala. Mencionar horario de consultas.

Notas de timing

Bloque	Filminas	Plan	Flexibilidad
Apertura	F-01	5 min	No reducir — establece el tono
Bloque 1	F-02 ... F-13 (12)	20 min	Reducir discusión final (F-13) si hay atraso
Bloque 2	F-14 ... F-20 (7)	25 min	F-18 (cuello de botella) puede resumirse a 2 min
Bloque 3	F-21 ... F-24 (4)	20 min	Bloque más comprimible — mostrar solo C si aprieta el tiempo
Bloque 4	F-25 ... F-39 (15)	30 min	F-29–F-32 (lógico) se pueden comprimir a 2 min si hay atraso
Bloque 5	F-40 ... F-45 (6)	15 min	Expandir si hay buena discusión, comprimir a 10 si hay atraso
Cierre	F-46 ... F-47 (2)	10 min	No reducir — adelanto de Clase 2 es importante
Total	47 filminas	125 min	= 120 min de clase + 5 min apertura

Señales de alerta durante la clase

- **Si el Bloque 2 consume más de 30 min:** saltar la comparación de lenguajes puros vs. multi-paradigma (se retoma en Tema 3)
 - **Si hay problemas con Deno Playground:** usar el TypeScript Playground oficial como fallback inmediato
 - **Si la demo de IA no funciona bien:** tener un screenshot preparado como backup
 - **Si los alumnos no tienen bagaje de C:** el Bloque 3 se puede dar solo con pseudocódigo, sin el snippet de LC-3
-

Referencias bibliográficas activas en esta clase

Autor	Obra	Capítulo	Usado en
Sebesta	<i>Concepts of Programming Languages</i> , 12th ed.	Cap. 1	Bloques 1, 2
Louden & Lambert	<i>Programming Languages: Principles and Practice</i> , 3rd ed.	Cap. 1	Bloques 2, 4
Gabbrielli & Martini	<i>Programming Languages: Principles and Paradigms</i> , 2nd ed.	Cap. 1	Bloques 3, 4
Schmidt & Runfola	<i>Liberating Logic in the Age of AI</i> (arXiv:2511.17696)	§2, Fig. 8, 12, 14	Bloque 5